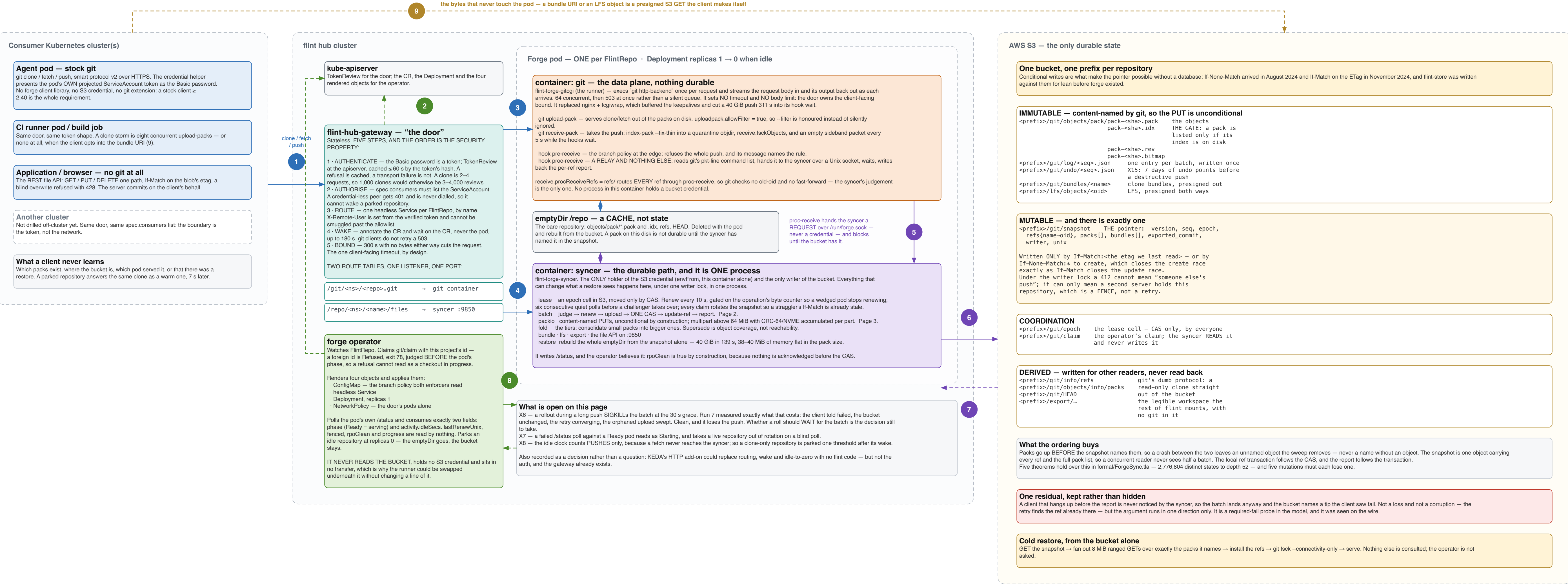


Components and data flow

A git server per repository: stock git does every git operation, S3 is the only durable state, and the syncer beside git is the only process that can write the bucket. Blue is the data plane (every byte of a clone or push), purple the durable path, green the control plane. Nothing on the data plane holds a bucket credential; the control plane never reads an object.



The nine flows

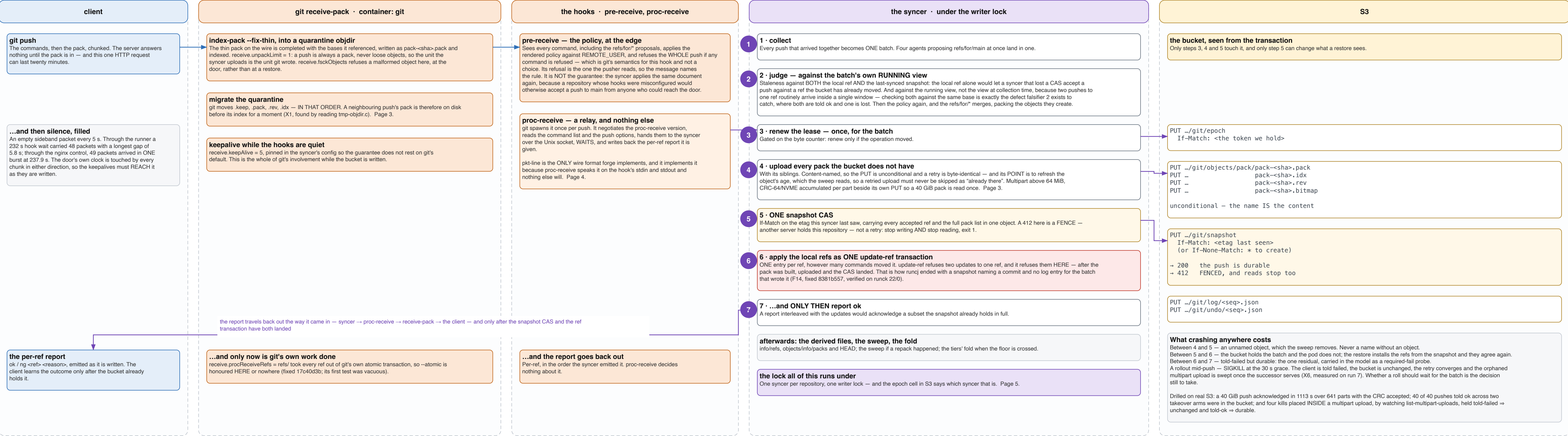
- clone / fetch / push over HTTPS; the Basic password is the pod's own projected SA token
- TokenReview at the apiserver, cached ≤ 60 s — the door authenticates BEFORE it wakes anything
- routed to the headless Service: the runner → git http-backend → upload-pack | receive-pack
- the REST file API — same auth, same consumers list, same wake, on the syncer's own port
- git upload-pack — serves clone/fetch out of the packs on disk
- git receive-pack — takes the push: index-pack --fix-thin into a quarantine objdir, receive.fsckObjects, and an empty sideband packet every 5 s while the hooks wait
- hook pre-receive — the branch policy at the edge; refuses the whole push, and its message names the rule
- hook proc-receive — A RELAY AND NOTHING ELSE: reads git's pkt-line command list, hands it to the syncer over a Unix socket and WAITS for the bucket
- the batch: every pack up first, then ONE snapshot CAS, then the local refs, then the report
- cold start, or a wake from replicas 0: restore the emptyDir from the snapshot alone
- the operator applies four objects and polls /status — it reads no object in the bucket
- bundle URI / LFS: presigned, client-to-store — the pod sees a few hundred bytes of JSON

Where the planes meet, one-directional at every joint

Control decides which bytes flow at all and makes the pod they will reach; control's lease produces the fence the durable path relies on, and its grace period decides whether a roll lands as a clean release or a SIGKILL mid-batch; the durable path writes /status and control believes it; and the data plane hands the durable path exactly one thing, through proc-receive, a request.

A push is one transaction, and the order of its steps is the argument

Under proc-receive, git's receive-pack serialises nothing and checks nothing for the commands it hands off — no old-oid check, no fast-forward test — and two concurrent pushes to one ref run their hooks fully overlapped. So the hook decides nothing. The syncer does: one process per repository, holding the writer lock for every path to S3.



The finding that shaped the whole path
Two concurrent pushes to one ref run their proc-receive hooks fully overlapped, and git checks nothing for them. So the hook cannot serialise and cannot decide; everything that must be decided once, in an order, is decided in the syncer under one lock, in one process.

How the files in a commit become a pack, and how the pack becomes an S3 object

Forge writes none of this format. git's own pack-objects builds it on the client, git's own index-pack completes and indexes it on the server, and what the syncer moves to S3 is the byte-for-byte file git wrote. The name of that file is the checksum of its own contents, which is the property the whole storage design leans on.

on the client — git's object database

A · the files in a commit are three kinds of object

Each file's CONTENT becomes a blob. Each directory becomes a tree, listing the mode, name and oid of every entry. The commit names one tree, its parents, an author, a committer and a message.
An object is stored as its type and length, a NUL, then the bytes — and its NAME is the SHA-1 (SHA-256 if the repository is) of that whole header-plus-body. Identical content anywhere in history is therefore the same object, stored once.

```
commit c3f9...
 tree   8a11...
 100644 README.md → blob e69d...
 040000 src      → tree 5b2c...
parent 9d02...      100644 main.rs → blob 4fa0...
```

B · loose on disk, packed on the wire

A new object is written LOOSE: zlib-deflated, one file, at .git/objects/e6/9de29b.... That is a fine store and a poor transport — no delta between versions of the same file, and a round of syscalls per object. So a push does not send loose objects; git runs pack-objects.

C · pack-objects, and the THIN pack

The two sides negotiate — "I have these tips, you have those." pack-objects then emits only the objects the server lacks, delta-compressed, and it is allowed to delta against a base the server ALREADY HAS but which is not in the pack. That is a thin pack: correct on the wire, unusable as a file.

What forge does NOT do to a pack

It does not parse it. It does not re-chunk it, deduplicate inside it, transcode it, or store objects individually, and it keeps no database of oids.

The consequences are the design:

- a pack in the bucket is a pack git can open, so a restore is a download and not a rebuild;
- the key is a checksum of the body, so two writers cannot disagree and a retry is free;
- the only thing needing a conditional write is the ONE object that says which packs and refs are current;
- and forge's correctness argument is about ORDERING and COORDINATION, not about a format it would otherwise have to get right.

The cost is that the unit of storage is git's unit. That is what the tiers, the fold cap and the pack-pinning question on runc1 are all about.

the pack format — what is actually in the file

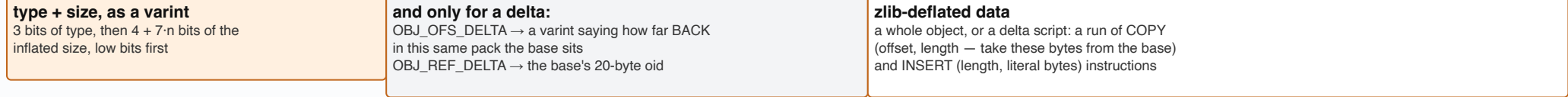
D · the .pack file, end to end

Every length is big-endian; the file is a byte stream with no padding and no index inside it.



THAT TRAILER IS THE NAME: git calls the file pack-<that sha>.pack, so the file name is a checksum of the file — which is why the S3 PUT can be unconditional, why a retry is byte-identical, and why two servers writing the same pack cannot disagree.

one object entry, expanded



the seven type codes

1 OBJ_COMMIT 2 OBJ_TREE 3 OBJ_BLOB 4 OBJ_TAG
6 OBJ_OFS_DELTA 7 OBJ_REF_DELTA (5 is unused)

A delta chain bottoms out at a full object, and git bounds the chain's depth so a read is not unbounded work.

core.bigFileThreshold = 1m

Objects above this skip the delta search in EVERY pack-objects, upload-pack's included. A 256 MiB blob tier cost 17 CPU-seconds per clone at git's 512m default, and 0.6 s at 1m.

E · on the server: index-pack --fix-thin, in a quarantine

receive-pack reads the thin pack into a temporary object directory, resolves every delta, APPENDS the bases the client left out so the file stands alone, and writes the .idx beside it. Only then does it migrate the quarantine into objects/pack/ — moving .keep, .pack, .rev, .idx, in that order.
receive.unpackLimit = 1 forces this path for every push: a push is always a pack, never loose objects, so the unit the syncer uploads is exactly the unit git wrote.

F · the .idx — the file that makes a pack readable

```
\377 t 0 c 4 B magic version 2 4 B
fanout[256] 4 B each fanout[b] = how many oids start ≤ b
oid[N] sorted, so a lookup is a bisect in one bucket
crc32[N] 4 B each over each object's PACKED bytes
offset[N] 4 B each MSB set → an index into the 8 B table
offset64[] 8 B each for packs over 2 GiB
pack sha 20 B ties this index to THAT pack
idx sha 20 B
```

THE INDEX IS A GATE, NOT A DETAIL — X1

The pack listing that feeds the upload step REQUIRES the pack's .idx to be on disk. Because git migrates the quarantine as .keep, .pack, .rev, .idx, a NEIGHBOURING push's pack is on disk before its index for a moment — and a batch that listed it then would have named, in a snapshot, a pack whose index never followed. A restore of that snapshot refuses: exit 78, unrecoverable.
Found by reading git's tmp-objdir.c, not by a test. Fixed by requiring the index; the mutation that removes the gate is one of the five the TLA+ model must lose.

the syncer's write — the pack becomes an object

G · the key is the name git already chose

```
<prefix>/git/objects/pack/pack-<sha>.pack
<prefix>/git/objects/pack/pack-<sha>.idx
<prefix>/git/objects/pack/pack-<sha>.rev
<prefix>/git/objects/pack/pack-<sha>.bitmap
```

No transcoding, no re-chunking, no container: the object's body IS the file's bytes.

H · and therefore the PUT is UNCONDITIONAL

The one place in flint where that variant is correct, and for the reason its doc comment gives. Content naming means a re-upload is byte-identical, so there is nothing to lose to a race — and its POINT is to refresh the object's age, which the sweep reads. A retried upload must therefore never be skipped as "already there".

I · ≤ 64 MiB is one PUT; above it, a multipart compose

put_whole holds the whole object in RAM, and a folded repository is ONE pack — the largest object forge ever writes — so 64 MiB is where the grid takes over. The part grid is contiguous from zero, covers the size exactly, uses at most max_parts, and makes every part but the last at least min_part: the rule S3 enforces, mirrored by the memory store, because a grid that is wrong only above 640 GiB is still wrong and EntityTooSmall from a real bucket is a poor place to discover it. The CRC-64/NVME that S3 judges at CompleteMultipartUpload is accumulated PER PART beside its own PUT, so a 40 GiB pack is read once. The ~70 s pre-pass it replaced ticked no progress and let the lease go quiet inside a live push.

J · nothing is durable until the snapshot names it

```
{"version":1, "seq":412, "epoch":7,
 "refs":{"refs/heads/main":"c3f9..."},
 "packs":["pack-1a2b...pack", "pack-9f0c...pack"],
 "bundles":[...], "exported_commit":"c3f9...",
 "writer":"forge-7f9-x2q", "unix":1757...}
```

PUT <prefix>/git/snapshot If-Match: <etag last seen>

K · later, the fold: many packs become fewer

A pack per push makes an unbounded list, so the tiers consolidate them — pack-objects writes a new pack, content-named like any other, and the snapshot's list is replaced. Supersede is OBJECT COVERAGE, not reachability, which is why unreachable objects from a refused merge can pin a whole pack. Measured on content that deltiflies: 81% of the snapshot-named bytes are residue — n=3, 81/81/81, and 82 on a fourth, differently-timed run. An earlier 58% came from one ~80 KB repository and read low.

L · and the way back

A restore reads the snapshot, fetches exactly the packs it names as 8 MiB ranged GETs at a bounded fan-out, drops them in objects/pack/, installs the refs, runs fsck --connectivity-only and serves. 40 GiB in 139 s from the delete, fsck clean, 25 MiB of anon RSS — because a pack from the bucket is a pack git can open, with no conversion at all.

The one place forge touches a wire format at all

pkt-line — four hex digits of length counting themselves, then the payload, and 0000 is a flush. That is the whole format, and forge implements it because proc-receive speaks it on the hook's stdin and stdout and nothing else will. Not the capability advertisement, not the ref advertisement, not the negotiation, not the sideband multiplexing, not the pack stream itself: stock git does every one of those, on both sides.

Forge does not interpret the git protocol; it runs the git server and stands behind it

The same shape — S3 as the only durable state, the disk a cache, one CAS'd pointer as the transaction boundary — was reached by several teams inside the same eighteen months. Where forge and its nearest neighbour walgit differ is in who owns the protocol: walgit reimplements receive-pack in Rust; forge lets git be the server and hooks in at one seam.

flint-forge — stock git, one seam

The client talks to git http-backend

A runner execs `git http-backend` once per request and streams the body in and the output back out. The runner implements no git: it is a CGI host. Swapping it for nginx + fcgiwrap and back again changed the chart, the CRD, the door's URL formula and every rig by nothing — which is the test of the claim.

Every git decision is made by git

The negotiation, the delta search, index-pack --fix-thin, the connectivity check, the ref advertisement, the sideband multiplexing, --filter, protocol v2, the reflog. A git bug is a git bug and a git release is an upgrade. When gitqual found uploadpack.allowFilter unset, the fix was one line of config, not a feature.

One seam, and it is a documented one: proc-receive

receive.procReceiveRefs = refs/ routes every ref through the hook. proc-receive negotiates its version, reads the command list, hands it to the syncer over a Unix socket, waits, and writes back the report. The acknowledgement the client sees IS the syncer's report, so "told ok" means the CAS landed — not that a disk was written. pkt-line is the ONLY wire format forge implements, and only because the hook's stdin and stdout speak it.

What that costs forge, honestly

A push's acknowledgement is a hook blocked on a Unix socket for as long as the bucket takes, so every party on the path needs an opinion about silence: receive.keepAlive = 5 s, a runner that does not buffer, a door whose only bound is 300 s of no bytes at all. Three of the campaign's four front-layer defects were in that chain — and all three were in nginx and fcgiwrap rather than in git or in the syncer.

walgit — the server reimplemented

The client talks to walgit

The server terminates the smart HTTP protocol itself and speaks receive-pack's half of it in Rust: the ref advertisement, the command list, the sideband, the report. upload-pack, repack and bundles are still stock git.

Every git decision on the write path is walgit's

Whatever receive-pack does — old-oid checks, the hook contract, the report's exact grammar, the keepalive discipline, quarantine semantics — has to be re-derived, and then kept in step with git as it changes. The payoff is that the writer can be exactly what the storage wants, with no hook waiting on a socket.

No seam — the writer IS the server

The acknowledgement is the server's own, which is simpler to reason about, and it is why walgit needs no keepalive story on the write path: nothing is waiting on a hook. Continuity states the same durability guarantee and does not say how it is carried.

What that costs walgit, honestly

The protocol surface is the thing that must not drift, and it is large. Forge's own measurement is the fair one to quote: after the tiers and the window, forge's worst push fell from 816 s to 0.83 s and its rate rose to 14.1 pushes/s against walgit's 10.3 — while walgit still uploads about a third of the bytes.

The seam, drawn as one sentence

client git → HTTPS → the door (authenticate, authorise, route, wake, bound) → the runner → git http-backend → git receive-pack → [pre-receive: the policy] → [proc-receive: pkt-line in, a request over a Unix socket, WAIT] → the syncer: judge, renew, upload the packs, ONE snapshot CAS, update-ref → the report back out through proc-receive → git writes the per-ref ok → the client.
Everything before the second bracket is stock git. Everything after it is forge.

What is convergent

"S3 as the only durable state, the on-disk repository a cache, one CAS'd pointer as the transaction boundary, stock git doing every git operation" is what Cursor published as Continuity on 2026-08-18, and what walgit — an open-source Rust server on that design — ships with bundle-uri, LFS and per-repository push policy. Behind both is Palantir's Stemma on JGit's DfsRepository, and Google's Bigtable-backed JGit before it; AWS CodeCommit stored packs in S3 with metadata in DynamoDB from 2015. Forge's design is dated 2026-09-04 and cites neither, because its author had not seen them. The SHAPE is not forge's claim.

What is forge's own — in the joints, not the shape

The writer is stock receive-pack with the syncer as its only client and the acknowledgement carried by proc-receive waiting on it. Coordination is a single-writer lease with a progress-gated renewer and a rotation on every claim, chosen over "any host may CAS" because at fleet rates every self-inflicted 412 would be a full restore. The control plane is Kubernetes-native per repository: the pod's own SA token as the credential, one pod parked at replicas 0, woken by a door that authenticates before it wakes. And the legible export republishes the repository as a workspace with no git in it, which nothing in the table attempts.

The verdict, stated so that it can be wrong

Innovative in the integration's joints — how the acknowledgement is carried, how one writer is kept and deposed, how the server is placed and woken, how the repository is re-published for readers that are not git — and convergent in its shape, which is now the shape of the field. A fifth thing is method rather than architecture: a TLA+ model whose mutations are the campaign's findings, eleven falsifiers, three campaigns on real S3 and a control arm that must fail, all in the repository, where Continuity's verification is production and walgit's is a seeded fault-injection simulation.

One writer per repository, and what the bucket looks like while it changes

One writer is a mechanism, not a convention: an epoch cell in S3 that every participant moves only by compare-and-swap. A renewing holder is undeposable by construction, so the whole design is about when it stops renewing — and about making a straggler's writes stale before its successor serves a byte.

a holder's life

- 1 • claim**
CAS the epoch cell. A 412 here means someone else got there first: wait, and poll again.
- 2 • ROTATE the snapshot**
The same content, a NEW etag, written BEFORE the successor restores — so any If-Match a straggler still holds is stale before the successor serves a byte. Every claim but a released cell's rotates, because only a releaser has proven it fenced itself before writing the mark (X12); and a repository nobody had published rotates too, by creating the empty snapshot (X11).
- 3 • sweep**
Orphaned multipart uploads, and unnamed packs from the predecessor's crash.
- 4 • restore**
The snapshot's packs into the emptyDir, the refs installed exactly, fsck --connectivity-only.
- 5 • serve, and renew every 10 s**
The renewer is its OWN task from the moment the claim lands, so the restore, every batch and every export beat through it. And it is gated: while a phase must progress, renew ONLY if the operation's byte counter advanced since the last renewal — because renewing for a wedged pod would trade a live pod losing its repository for a dead one keeping it forever, which is the case the quiet polls exist for.
- 6 • a 412 on any CAS of ours = THE FENCE**
Stop writing AND stop reading, exit 1. A deposed server that kept answering upload-pack would serve stale refs forever. The kubelet restarts the pod: a fence exits 1, a refusal exits 78, and restart is not an operator decision.

the challenger, and the fence

- the challenger**
Polls the epoch cell, and judges the holder dead only after SIX CONSECUTIVE polls saw the SAME token — never on one quiet poll, and never on a clock. A clean release on SIGTERM writes the mark and lets the successor claim at once, which is the difference between a rollout and a crash.
- the sensor can lie, and it did**
The renewer's gate reads a byte counter. Run 3's checksum pre-pass did 70 seconds of real work and ticked nothing, so the counter said "no progress" while the process was busy. The TLA+ model keeps the SENSOR and the REAL MOVEMENT as separate variables for exactly this reason.
The renewer's first shape renewed only inside a push, and only between an upload's parts: on runbw a 10 GiB push's token was silent for 125 s and a restore's for 141 s, inside a 60 s window. A challenger claimed the live, importing pod 62 s after arriving, and the two seized each other through three epochs — while every acknowledged push still reached the bucket.
- what the model found that the code had wrong**
The first shape exempted two cases from the rotation, and the model's first two strict runs refuted both against the code the same day.
X11 — a takeover of a repository nobody had published skipped the rotation, on the theory that the first CAS's If-None-Match would be the fence. But a straggler mid-batch on the old epoch could land its create after the successor served, and then the SUCCESSOR's own first CAS was what 412'd, fencing it with its predecessor's push.
X12 — a successor that died between its takeover CAS and its rotation came back through self-recognition, which skipped the rotation because "our own previous process died with its writes" — while the straggler from the epoch before still held a valid If-Match.
Both are now unit tests that fail against the old code.
- what the drills measured**
On runbx a challenger sat beside a live 40 GiB restore for 398 s and never claimed, the holder was never fenced, and 24 of 24 pushes under that contention were durable.
Two servers for one repository is not a state the Deployment creates on its own: it takes a roll against a wedged pod, a lost node whose pod is still counted, or a hand — and every such case ends with the straggler's next CAS 412ing.

the bucket over time — what accumulates, what is removed, and by whom

- the packs**
Added by every batch, content-named, never meaningfully overwritten. Removed only by the sweep, and only when the snapshot no longer names them AND they are older than the grace. A re-upload REFRESHES the object's age, which is why the PUT must not be skipped as redundant.
- the snapshot**
One object, rewritten by CAS on every batch and on every claim's rotation. seq is monotonic so that a rotation changes the bytes — and therefore the etag — even when the content would be identical. version is bumped only for a change an older syncer could MISREAD, and a reader meeting a higher version refuses rather than parsing what it can and concluding the repository is empty.
- the fold, and the tiers**
A pack per push is an unbounded list, so the tiers consolidate. The cadence used to repack the WHOLE repository every 24 pushes — 33x the bytes pushed, and one push held for 816 s. The tiers with a floor replaced it: 1.67–1.83x against 2.90–4.33x without, measured on a cluster. Supersede is object coverage, not reachability, so dead objects can pin live packs — 81% of the snapshot-named bytes are residue, n=3, 81/81/81.
- the log, and the undo points**
log/<seq>.json: one immutable entry per batch, written once, so a follower may cache what it reads.
undo/<seq>.json: before a batch that would make a state unreachable, the snapshot it is about to replace is kept for 7 days and both sweeps know it (X15, --undo-list).
- what the sweep must never do**
Delete an object a snapshot still names, or an object younger than the grace — a pack uploaded in step 4 of a batch whose CAS has not landed yet is exactly that. A fold running while a sweep runs is a modelled case, not a hoped-for one.

The quantitative axiom, stated because it cannot be discharged

The model proves the renewer's sensor honest ONLY under the assumption that the challenger's polls and the holder's heartbeats share a period. That is a real-time property no model checker of this kind can establish; it is a configuration invariant the chart must keep true, and it is written down here so that it can be checked rather than assumed. The other edges, kept and named: told-failed-but-durable; a rollout mid-push losing the push; the export as a mirror nobody repairs; the push-only idle clock; and the blind poll that reads a Ready pod as Starting.